42 · COMMON CORE · C++

Module 01
Task Guide

Memory allocation, pointers to members, references and switch statements — what each of the seven exercises is really asking, the concept it is built to teach, and the traps that cost points at defense.

Companion to `c++-01.pdf` · subject version 11.1
Standard: C++98 · Flags: `-Wall -Wextra -Werror`

How to use this document. It explains and it questions — it does not hand you finished exercises. The reasoning is the part being graded, so every section ends with the questions an evaluator can ask you.

CONTENTS

What is in this guide

§1	The rules that fail you	grading landmines before any code
§2	Concept primer	stack vs heap, references, member pointers
ex00	BraiiiiiiinnnzzzZ	when to allocate on stack vs heap
ex01	Moar brainz!	array allocation, `new[]` / `delete[]`
ex02	HI THIS IS BRAIN	references demystified
ex03	Unnecessary violence	reference member vs pointer member
ex04	Sed is for losers	C++ streams and `std::string`
ex05	Harl 2.0	pointers to member functions
ex06	Harl filter	the `switch` fallthrough trick
§3	Pre-defense checklist	what to verify before you push

SECTION 1

The rules that fail you

These are not style suggestions. Each one carries a stated penalty in the subject, and an evaluator checks them before looking at whether your code works.

RULE	PENALTY
`printf`, `malloc`, `calloc`, `realloc`, `free` — any of them	**0** — "and that's it"
`using namespace std;` or the `friend` keyword	**−42**
STL containers (`vector`, `list`, `map`…) or `<algorithm>`	**−42** (allowed only from module 08)
A function body written inside a header (templates excepted)	**0** on that exercise
Missing include guards	**0**
External libraries, C++11 or later features, Boost	forbidden

Compilation

```
c++ -Wall -Wextra -Werror -std=c++98 *.cpp -o program
```

`-Werror` means every warning is fatal. If it compiles for you but the evaluator adds `-std=c++98` and it breaks, that is on you — so always compile with the flag yourself.

Makefile

Same rules as the C projects: the mandatory rules are `$(NAME)`, `all`, `clean`, `fclean`, `re`. It must not relink — running `make` twice in a row should say there is nothing to be done. Do not use a wildcard to list your sources; name them explicitly.

Things people forget

- Every output message ends with a newline and goes to **standard output**.
- Each header must compile on its own — it includes what it needs, and nothing more.
- Orthodox Canonical Form is required from module 02 onwards, so it is *not* demanded here — but writing a destructor that actually cleans up is.
- No memory leaks. Every `new` has exactly one matching `delete`.
- Directory names are exactly `ex00` … `ex06`; file names must match the subject character for character.

SECTION 2

Concept primer

Four ideas carry the whole module. If these are solid, the exercises stop being puzzles.

2.1 Stack vs heap

	STACK	HEAP
How	`Zombie z("Foo");`	`Zombie* z = new Zombie("Foo");`
Lifetime	Ends at the closing brace of its scope — automatically	Lives until *you* write `delete`
Destructor	Called for you, guaranteed	Called only by `delete`
Cost	Essentially free	A real allocation, and a leak if you forget
Use when	The object is used and finished inside one function	The object must outlive the function that made it, or its size/count is only known at runtime

The whole point of ex00 is to make you feel this difference, so do not skim it. Returning the address of a stack object from a function is the classic disaster: the object is destroyed at `return`, and the caller holds a pointer to memory that no longer belongs to it.

2.2 References vs pointers

A reference is another name for an existing object. Under the hood the compiler does the same address bookkeeping a pointer does — the difference is what the language lets you write.

	POINTER `T*`	REFERENCE `T&`
Can be null	Yes — so it can mean "nothing yet"	No, never
Can be reseated	Yes, point it elsewhere any time	No — bound once, at initialisation, forever
Must be initialised	No	Yes, at the moment of declaration
Syntax to use it	`*p`, `p->member`	Just the name, like the original

That table *is* the answer to ex03's question. A member that must always be present and never change target wants a reference. A member that may be absent, or replaced later, needs a pointer — because only a pointer can express "none" and "now this one instead".

A class with a reference member must bind it in the constructor's **initialisation list**, not in the body:

```
class Example {
    private:
        std::string  _name;
        Thing&       _thing;   // a reference member
    public:
        Example(std::string n, Thing& t);
};

// in the .cpp — bound here, before the body runs:
Example::Example(std::string n, Thing& t) : _name(n), _thing(t) { }
```

2.3 Returning a const reference

```
const std::string&  getType(void) const;
```

Two separate things are happening. Returning `&` means no copy of the string is made — the caller sees the live attribute, which is exactly why an update through `setType()` is visible to a holder that fetched it earlier. Returning `const` means the caller can read it but cannot assign through it and bypass your setter. The trailing `const` promises the function does not modify the object.

2.4 Pointers to member functions

A member function is not a plain function: calling it requires an object. So its pointer type names the class, and calling it requires an instance and a special operator.

```
// declare: pointer to a member of Foo taking void, returning void
void  (Foo::*fp)(void);

// assign: & and the class name are both mandatory
fp = &Foo::someMethod;

// call, on an object            / through a pointer to one
(obj.*fp)();                      (ptr->*fp)();
```

The parentheses around `obj.*fp` are required — `.*` binds looser than the call `()`. Because these pointers all share one type, you can hold several in an array and pick one by index, which is the mechanism ex05 wants instead of an if/else chain.

2.5 switch

`switch` only accepts an integral or enumeration value — never a `std::string`. So the string level has to be turned into a number first. And a `case` without `break` falls through into the next one, which is normally a bug but in ex06 is precisely the feature you want.

EXERCISE 00

EXERCISE 00

BraiiiiiiinnnzzzZ

DIRECTORY	`ex00/`
FILES	`Makefile`, `main.cpp`, `Zombie.{h,hpp}`, `Zombie.cpp`, `newZombie.cpp`, `randomChump.cpp`
FORBIDDEN	None

What is asked

A `Zombie` class with a private `std::string name` and a member function `void announce(void);` printing exactly:

```
<name>: BraiiiiiiinnnzzzZ...
```

with the real name substituted and no angle brackets. Then two free functions, each in its own file: `Zombie* newZombie(std::string name);` creates a zombie, names it and returns it so it survives outside the function; `void randomChump(std::string name);` creates a zombie, names it, and makes it announce itself. The destructor must print the zombie's name.

THE REAL POINT

The two functions are deliberately near-identical so that one difference stands out: one hands an object back to the caller, the other uses it and is done. That difference decides the allocation. Work out which is which, and be able to say *why* — that sentence is the exercise.

How to think about it

- `newZombie` returns a `Zombie*`. Ask: does the object it points at still exist after the function returns? Only one of stack/heap answers yes.
- `randomChump` returns nothing and the zombie is never mentioned again. If it is on the stack, who destroys it, and when? Count the `delete` calls you need: zero.
- The printing destructor is your instrument. Run the program and read the order of the messages — that is the lifetime of each object, made visible. This is the whole lesson.

TRAPS

- Returning the address of a local object from `newZombie`. It compiles (maybe with a warning, which `-Werror` turns fatal) and the memory is dead on arrival.
- Forgetting `delete` in `main` for whatever `newZombie` returned — the destructor message simply never prints, which is your leak announcing itself.
- Getting the announce string wrong: three `i`… count them from the subject, and end with a newline.
- Setting the name after construction instead of in the constructor. Nothing forbids it, but a constructor taking the name is cleaner and is what the two functions want.

BE READY TO ANSWER

- Why did you allocate on the heap in one function and the stack in the other?
- What exactly happens at the closing brace of `randomChump`?
- Who is responsible for deleting what `newZombie` returns, and how would a caller know?

EXERCISE 01

EXERCISE 01

Moar brainz!

DIRECTORY	`ex01/`
FILES	`Makefile`, `main.cpp`, `Zombie.{h,hpp}`, `Zombie.cpp`, `zombieHorde.cpp`
FORBIDDEN	None

What is asked

`Zombie* zombieHorde(int N, std::string name);` must allocate **N zombies in a single allocation**, give each of them the name, and return a pointer to the first. Then call `announce()` on each and free everything.

THE REAL POINT

"A single allocation" rules out a loop of N separate `new` calls. One block, N objects — which means `new Zombie[N]`, and therefore `delete[]`, and therefore a default constructor, because array-new cannot pass arguments in C++98. Each constraint follows from the previous one; noticing that chain is the exercise.

How to think about it

- Because `new Zombie[N]` default-constructs every element, the names have to be set *after* allocation. So the class needs a way to be named later — plan for it.
- The return is a pointer to the first element. The horde is contiguous, so the caller walks it with an index or by advancing a pointer.
- Consider what `N <= 0` should do. The subject does not specify, so decide and be able to defend your choice.
- Your own `main` is graded here — "implement your own tests". Show a horde being created, announcing, and being freed, with the destructor messages proving it.

TRAPS

- `delete` instead of `delete[]`. Undefined behaviour: only the first destructor runs and the block is freed wrongly. This is the single most common failure here.
- Allocating in a loop. It works, it leaks nothing, and it still fails the exercise, because the subject says one allocation.
- Deleting an interior pointer. If you advanced the returned pointer while iterating, you no longer hold the address you must pass to `delete[]` — keep the original.
- Adding a constructor with arguments and no default constructor: `new Zombie[N]` then refuses to compile. Both must exist if you want both call styles.

BE READY TO ANSWER

- Why `delete[]` and not `delete` ? What does the extra bracket change?
- How does the runtime know how many destructors to run?
- Why can't you pass the name straight to `new Zombie[N]` ?

EXERCISE 02

EXERCISE 02

HI THIS IS BRAIN

DIRECTORY	`ex02/`
FILES	`Makefile`, `main.cpp`
FORBIDDEN	None

What is asked

One `main.cpp`. A `std::string` holding `"HI THIS IS BRAIN"`, a pointer to it called `stringPTR`, a reference to it called `stringREF`. Print the three addresses, then the three values. Six lines. That is the entire exercise.

THE REAL POINT

The output is the lesson: all three addresses are identical, and all three values are identical. A reference is not a new object and not a copy — it is the same memory under a second name. Once you have seen the three matching addresses, references stop being mysterious.

How to think about it

- The address of the variable itself is `&variable`. The address *held by* the pointer is the pointer's own value. The address "held by" the reference is `&stringREF` — taking the address of a reference gives you the address of the referred-to object, which is the point being made.
- Use `std::cout`, not `printf` — `printf` is an instant zero.
- Declare the reference with `std::string& stringREF = variable;`. It must be initialised right there; a reference cannot be declared empty and bound later.
- Label your six lines so the reader can tell which is which. Unlabelled addresses prove nothing.

TRAPS

- Printing `stringPTR` when you meant the value, or `*stringPTR` when you meant the address. Read your own output and check it says what you think.
- Expecting `&stringREF` to give you "the address of the reference". It does not — and understanding why is exactly what the exercise is testing.

BE READY TO ANSWER

- Why are all three addresses the same?
- Name three things a pointer can do that a reference cannot.
- Does a reference occupy memory of its own?

EXERCISE 03

EXERCISE 03

Unnecessary violence

DIRECTORY	`ex03/`
FILES	`Makefile`, `main.cpp`, `Weapon.{h,hpp}`, `Weapon.cpp`, `HumanA.{h,hpp}`, `HumanA.cpp`, `HumanB.{h,hpp}`, `HumanB.cpp`
FORBIDDEN	None

What is asked

`Weapon` holds a private `std::string type`, returns it from `getType()` as a **const reference**, and can change it via `setType()`. `HumanA` and `HumanB` each have a name and a weapon and an `attack()` printing:

```
<name> attacks with their <weapon type>
```

They differ in exactly two ways: HumanA receives the weapon in its constructor and is always armed; HumanB does not, gets one later through `setWeapon()`, and may have none.

THE REAL POINT

The subject's `main` creates a weapon, arms a human, attacks, calls `setType()` on the *weapon*, and attacks again — and the second attack must show the new type. That only happens if the human refers to the original weapon instead of holding a copy of it. Two humans, two ways of referring: one that can never be absent (reference) and one that can (pointer). The exercise is the contrast.

How to think about it

- **HumanA** is always armed and its weapon is fixed at construction. Never null, never reseated — a reference member, bound in the constructor's initialisation list.
- **HumanB** may exist with no weapon at all, and gains one later. "No weapon" and "a different weapon now" are both states only a pointer can represent. Initialise it to `NULL` (not `nullptr` — that is C++11) and have `attack()` check before dereferencing.
- Storing a `Weapon` by value in either class would break the required output: the human would attack with a stale copy. Verify by running the subject's `main` unmodified.
- Nothing is allocated with `new` here, so there is nothing to `delete`. Being able to say that — and why the subject still asks about leaks — is worth points.

TRAPS

- Assigning a reference member in the constructor body. It will not compile; references bind in the initialisation list.
- HumanB dereferencing a null weapon. Decide what an unarmed HumanB prints, and handle it.
- Returning `std::string` by value from `getType()` — the subject says const reference, and by-value quietly reintroduces the stale-copy bug.
- Include cycles between the headers. `HumanA.hpp` needs `Weapon.hpp`; `Weapon.hpp` needs neither human. Keep the dependency one-way and guard every header.

BE READY TO ANSWER

- Why a reference in HumanA and a pointer in HumanB? Could you swap them?
- Why does the second `attack()` print the updated weapon type?
- What would break if `getType()` returned by value?
- What happens if the `Weapon` dies before the `HumanA` that refers to it?

EXERCISE 04

EXERCISE 04

Sed is for losers

DIRECTORY	`ex04/`
FILES	`Makefile`, `main.cpp`, `*.cpp`, `*.{h,hpp}`
FORBIDDEN	`std::string::replace` — and all C file functions

What is asked

A program taking three arguments: a filename, `s1` and `s2`. Read `<filename>`, write `<filename>.replace` containing the same content with every occurrence of `s1` replaced by `s2`. C file functions (`fopen`, `fread`, …) count as cheating. Every `std::string` member is allowed *except* `replace`. Handle bad input and errors, and turn in your own tests.

THE REAL POINT

Two things at once: doing file I/O the C++ way with `<fstream>`, and building the substitution by hand out of `find`, `substr` and concatenation. Forbidding `replace` forces you to actually think about indices instead of calling one function.

How to think about it

- The tools you need are `std::ifstream`, `std::ofstream`, `std::getline`, and from `std::string`: `find`, `substr`, `length`, `+`, and `std::string::npos`.
- `find` returns `npos` when there is no match — that is your loop's exit condition. Rebuild the output as "text before the match" + `s2` + "keep searching after the match".
- Search must resume *after* the inserted `s2`, not at the start. If `s2` contains `s1`, restarting from zero loops forever. Test that case deliberately.
- Newlines: `getline` strips the delimiter. Decide how you re-emit them so the copy is not missing or gaining a trailing newline. Reading the whole file with a stream buffer instead of line by line is another valid route — either way, be deliberate.
- Error cases to handle: wrong number of arguments, file that does not open, output that cannot be created, and an empty `s1` (searching for nothing). Print a clear message and return a non-zero status.

TRAPS

- Infinite loop when `s2` contains `s1` — for example replacing `a` with `aa`.
- Not checking that the input stream actually opened, then producing an empty `.replace` file and reporting success.
- Reaching for `std::string::replace` out of habit. Explicitly forbidden.
- Comparing your output to `sed` only on files that end in a newline. Test one that does not.

BE READY TO ANSWER

- What does `find` return when there is no match, and what type is it?
- Walk through your loop with `s1 = "a"`, `s2 = "aa"`.
- What happens with an empty file? A file with no occurrence at all? A file you cannot read?

EXERCISE 05

EXERCISE 05

Harl 2.0

DIRECTORY	`ex05/`
FILES	`Makefile`, `main.cpp`, `Harl.{h,hpp}`, `Harl.cpp`
FORBIDDEN	None

What is asked

A `Harl` class with four **private** member functions — `debug`, `info`, `warning`, `error`, each taking and returning `void` — and one public `void complain(std::string level);` that calls the right one. The subject supplies the four complaint texts; you may use your own.

THE REAL POINT

The subject is blunt: "The goal of this exercise is to use pointers to member functions. This is not a suggestion." A working `if/else if/else` chain is a failed exercise even though the output is identical. You are being graded on the mechanism, not the result.

How to think about it

- All four functions share one signature, so one pointer type describes all of them: `void (Harl::*)(void)`. That is what makes an array of them possible.
- Keep two parallel arrays — the four level names, and the four member pointers — then find the index of the incoming string in the first and call the pointer at that index in the second. The lookup is one small loop; no branching per level.
- The call syntax inside a member function of `Harl` is `(this->*array[i])();`. The parentheses are not optional.
- Decide what `complain()` does with a level that matches nothing. The subject is silent here (ex06 defines it), so pick a sane behaviour and be able to justify it.
- The four functions being private is deliberate: `complain` is the only way in.

```
// the shape of the tool — not the exercise, just the syntax
typedef void (Harl::*HarlMethod)(void);

HarlMethod  table[4] = { &Harl::debug, &Harl::info,
                         &Harl::warning, &Harl::error };
// ... having found the matching index i:
(this->*table[i])();
```

TRAPS

- Writing the if/else chain "just to get it working" and never coming back. It is the one thing the subject explicitly forbids.
- Forgetting `&Harl::` when taking the address — the class name and the ampersand are both required.
- Dropping the parentheses: `this->*table[i]()` does not parse as you intend.
- Using `void*` or a plain function pointer. A member function pointer is a distinct type and is not convertible to either.

BE READY TO ANSWER

- Why is a member function pointer not the same as a normal function pointer?
- What does `this->*` actually do?
- How would you add a fifth level? How many lines does that cost you?

EXERCISE 06 · BONUS

EXERCISE 06 — OPTIONAL

Harl filter

DIRECTORY	`ex06/`
FILES	`Makefile`, `main.cpp`, `Harl.{h,hpp}`, `Harl.cpp`
EXECUTABLE	`harlFilter` — the name is mandatory
FORBIDDEN	None

What is asked

A program taking one level as an argument and printing that level **and every level above it**, each preceded by its header:

```
$> ./harlFilter "WARNING"
[ WARNING ]
I think I deserve to have some extra bacon for free.
I've been coming for years, whereas you started working here just last month.

[ ERROR ]
This is unacceptable! I want to speak to the manager now.

$> ./harlFilter "I am not sure how tired I am today..."
[ Probably complaining about insignificant problems ]
```

The subject requires you to use a `switch`. You can pass the module without this exercise.

THE REAL POINT

"One of the most effective ways to deal with Harl is to SWITCH him off." Levels are ordered, and a `switch` whose cases deliberately *omit* `break` falls through from the matched case down to the end — printing exactly "this level and everything more severe". The bug everyone is taught to avoid is here the whole design.

How to think about it

- `switch` cannot take a `std::string`. Convert the argument to an index first — the same name-lookup you already wrote in ex05 — then switch on that integer.
- Order the cases from least to most severe: DEBUG, INFO, WARNING, ERROR. Matching DEBUG then falls through all four; matching WARNING falls through two.
- An unrecognised level is the `default` case, printing the "insignificant problems" line. Also decide what happens with no argument, or more than one.
- Reuse the `Harl` from ex05 — copy the directory and extend it. The header names are the same on purpose.
- Match the expected output character for character: square brackets, spaces inside them, uppercase level names.

TRAPS

- Leaving `break` in every case — you then print one level only, and the exercise does nothing.
- Trying to `switch` on the string directly. It will not compile.
- Rebuilding the whole thing with if/else because fallthrough feels wrong. The subject asks for the switch by name.
- Not handling `argc != 2`.

BE READY TO ANSWER

- Why can't `switch` take a `std::string`?
- Explain fallthrough. Why is it a bug most of the time and correct here?
- What does your program do with no arguments, or with `"warning"` in lowercase?

SECTION 3

Pre-defense checklist

Per exercise

- Directory named exactly `exNN`, file names exactly as the subject lists them.
- `make` builds clean, `make re` works, running `make` twice does not relink.
- Compiles with `-Wall -Wextra -Werror -std=c++98` and produces zero warnings.
- Output matches the subject exactly, ends with newlines, goes to stdout.
- Every header has an include guard and compiles standalone.
- No function bodies in headers.

Memory

Every `new` is matched by `delete`, every `new[]` by `delete[]`. Verify rather than assume — on Linux:

```
valgrind --leak-check=full ./program
```

On macOS, `leaks --atExit -- ./program`, or build with `-fsanitize=address` and run. Also read your destructor messages: the count of construction messages must equal the count of destruction messages.

Grep your own repository

```
grep -rnE 'printf|malloc|calloc|realloc|free\(|using namespace|friend' ex0*
grep -rn '#include <\(vector\|list\|map\|algorithm\)>' ex0*
```

Both should come back empty. Each hit is a zero or a −42.

Understanding — the part that is actually graded

The subject warns that a brief modification may be requested live at defense: a small behaviour change, a few lines to rewrite, an easy feature to add. That is unfaked. If you can answer the "be ready to answer" questions in this guide out loud, without notes, you are ready. If you cannot answer them for an exercise, that exercise is not finished yet regardless of what it prints.

A WORD ON HOW TO USE THIS

This guide explains what each task asks and why, and points at the ideas and the traps — it deliberately stops short of writing the exercises for you. That is not caution for its own sake: the module is graded on reasoning you can defend live and later reproduce in an exam with no tooling at all. Struggle through a section, then use it to check whether your reasoning holds.